ARDUINO PHYSICAL AI CHALLENGE INDIA 2026

Project Report

Organised by Robu.in × Arduino · Submit as PDF · Max 10 MB

Project Title	Arduino UNO Q based Opensource AI Drone Platform
Team Name	JRFCQAI (Maker / Independent Developer -Solo Category)
Registration / Team ID	APC-2026-GJ-92321
Contest Track	Robotics, gaming and Interactive AI
Institution & City	GEC Gandhinagar, Gandhinagar - Gujarat

Team Members (1–4)

Name	Email	Role
BUTANI RAVI CHANDULAL	ravi.butani03@gmail.com	Maker / Independent Developer - Solo

1. Project Overview

Problem Statement

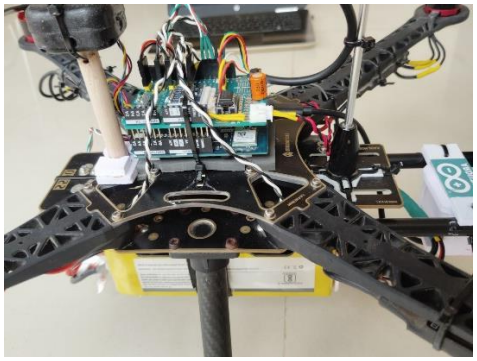
Most DIY and open-source autonomous drone platforms rely on a standalone microcontroller (such as an STM32) strictly for real-time flight control. To add vision-based artificial intelligence, builders traditionally mount an external single-board computer (SBC) like a Raspberry Pi. This dual-board configuration introduces major design drawbacks:

- Excessive total takeoff weight (AUW) and space requirements.
- High power consumption and complex wiring harnesses.
- Inter-device communication latency over physical serial connections.

While the new Arduino UNO Q provides a combined dual-brain architecture (STM32U5 + Qualcomm Dragonwing QRB2210 CPU), the board is currently new in market with limited documentation. Existing online projects use the UNO Q merely for basic ground-based robots. No open-source project has yet leveraged this board to run a full-fledged flight controller alongside a vision AI engine on the same single board. So I have developed JRFCQAI that utilizes dual brain architecture of UNO Q for flight control and edge AI flight computer.

Github Repo of the project : https://github.com/butaniravi/JRFCQAI

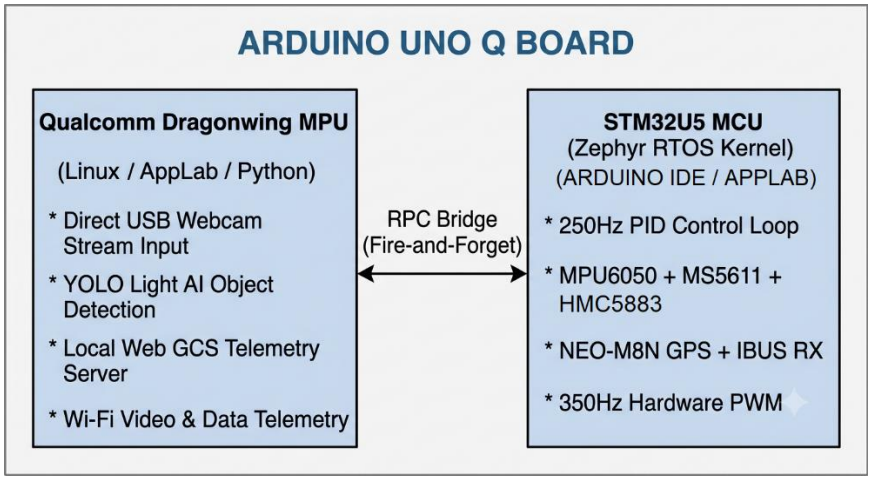
Youtube Demo Video and Playlist :
https://www.youtube.com/watch?v=u1FR_EWiBVY
https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4


How Your Project Works

JRFCQAI is an open-source hardware and software framework that transforms the Arduino UNO Q into a unified flight computer and live ground processing hub.

- **Real-Time Control Subsystem (STM32U5 Microcontroller):** Runs a ported YMFC-32 Autonomous flight program operating on top of the Zephyr RTOS kernel. It reads onboard sensors via I2C, decodes RC input via UART, and executes a deterministic 250Hz (4ms) PID loop to control four brushless motor ESCs over hardware PWM.
- **Artificial Intelligence & Telemetry Subsystem (Qualcomm Dragonwing CPU):** Executes a Python script inside Arduino AppLab. It captures frames from a USB webcam, runs a lightweight object detection AI Brick (YOLO Light), overlays bounding boxes on the video stream, and hosts a local web server.
- **Cross-Brain Synchronization:** Telemetry is transferred from the STM32U5 to the Qualcomm CPU using non-blocking, zero-latency RPC notification calls (bridge.notify() and brifge.provide()).
- **Ground Control Station (GCS):** Any device (smartphone, tablet, or laptop) connected to the board's local Wi-Fi network can open a standard web browser to view live flight telemetry, AI-processed video, and issue flight commands without requiring custom software installation.



Github Repo : https://github.com/butaniravi/JRFCQAI
Youtube Playlist of project Demos : https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4

Why Arduino UNO Q?

The Arduino UNO Q provides a dual-brain architecture that combines deterministic, real-time control with high-level Linux processing:

- High-Performance Microcontroller (STM32U585): Guarantees jitter-free execution of safety-critical PID stabilization loops and sub-millisecond sensor updates.
- Application Processor (Qualcomm Dragonwing QRB2210): Handles high-throughput tasks including USB camera video processing, AI model inference, and web hosting.
- Single-Board Efficiency: Eliminates the weight, cost, and failure points associated with running separate flight controller boards and companion single-board computers.

2. Components Used (BOM)

Component	Quantity	Purpose / Interface Details
Arduino UNO Q (ABX00087)	1	Dual-brain flight computer & AI processing engine
Custom General-Purpose Expansion PCB	1	Custom baseboard designed to route QWIIC, UARTs, PWM, and power distribution cleanly
MPU6050 Accelerometer & Gyroscope	1	Primary attitude estimation; connected via I2C1 (QWIIC) at 400kHz
MS5611 Barometric Pressure Sensor	1	High-precision altitude estimation; connected via I2C1 (QWIIC) at 400kHz
HMC5883L Magnetometer	1	Compass heading calculation; connected via I2C1 (QWIIC) at 400kHz
AT24C256 I2C EEPROM	1	32KB I2C EEPROM to hold calibration data of IMU Compass and baro; connected via I2C1 (QWIIC) at 400kHz
u-blox NEO-M8N GPS Module	1	Latitude/Longitude position holding and RTH; connected via UART3 at 115200 baud

Github Repo : https://github.com/butaniravi/JRFCQAI
Youtube Playlist of project Demos : https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4

FlySky FS-iA6B 10-Channel Receiver	1	Manual pilot override; connected via UART1 using iBUS protocol at 115200 baud
30A BLDC ESCs (Hardware PWM support)	4	Motor velocity control; driven by hardware PWM pins (D3, D5, D6, D9) at 350Hz
920kV BLDC Motors + 1045 Propellers + S500 Frame	4	Airframe propulsion (Quadcopter configuration)
3S 6000mAh 35C LiPo Battery	1	Primary system power source for motors and electronics
5V / 500mA Buck Converter	1	External power supply module for the USB webcam
Standard USB Webcam	1	Onboard video capture module; interfaced directly to UNO Q USB Type-C port
100µF 50V Low-ESR Capacitor	1	Power line filter placed across VIN to absorb motor back-EMF noise
Voltage Divider (10kΩ / 1kΩ Resistors)	1	Battery monitoring circuit connected to analog pin A0
Status LEDs (Red, Green, Blue)	3	Visual indication of system state; connected to digital pins D11, D12, and D13

3. System Architecture & Circuit

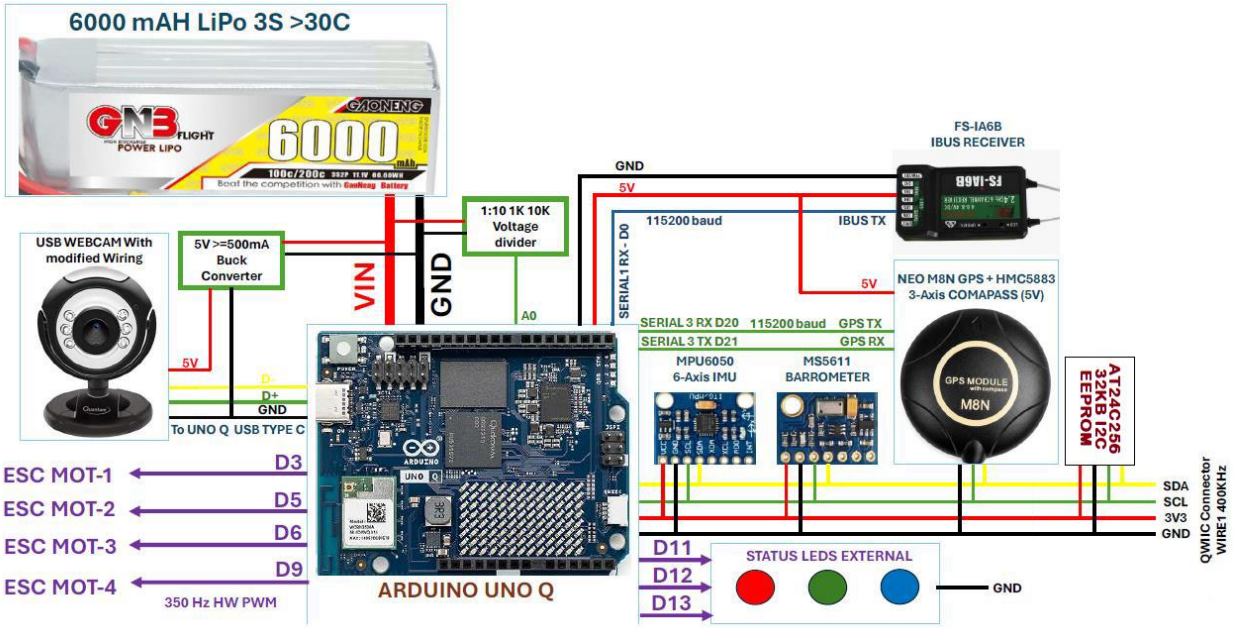
Step-by-Step Workflow

1. **Power Stabilization Phase:** 3S LiPo battery power passes through a 100µF filtering capacitor directly to the UNO Q VIN pin, while an auxiliary buck converter powers the onboard webcam.
2. **Real-Time Data Acquisition:** The STM32U5 microcontroller reads IMU, barometer,

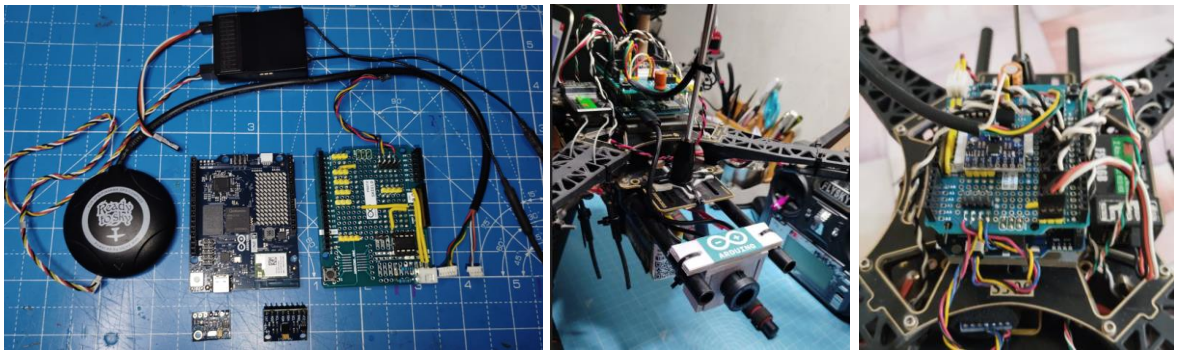
ATC24C256 EEPROM and magnetometer sensors over I2C1 at 400kHz, while parsing 10-channel iBUS RC data (UART1) and GPS coordinates (UART3).

3. **Flight Stabilization Execution:** The STM32U5 executes a deterministic 250Hz PID control loop under Zephyr RTOS, updating 350Hz hardware PWM outputs (pins D3, D5, D6, D9) to adjust motor speeds.
4. **Inter-Processor Synchronization:** The STM32U5 sends telemetry frames to the Qualcomm Dragonwing processor using non-blocking bridge.provide() and bridge.notify() RPC calls.
5. **AI Inference & Video Processing:** The Qualcomm CPU receives camera frames, executes the YOLO Light detection model inside Python AppLab, overlays visual bounding boxes, and updates the local Ground Control Station web page over Wi-Fi.

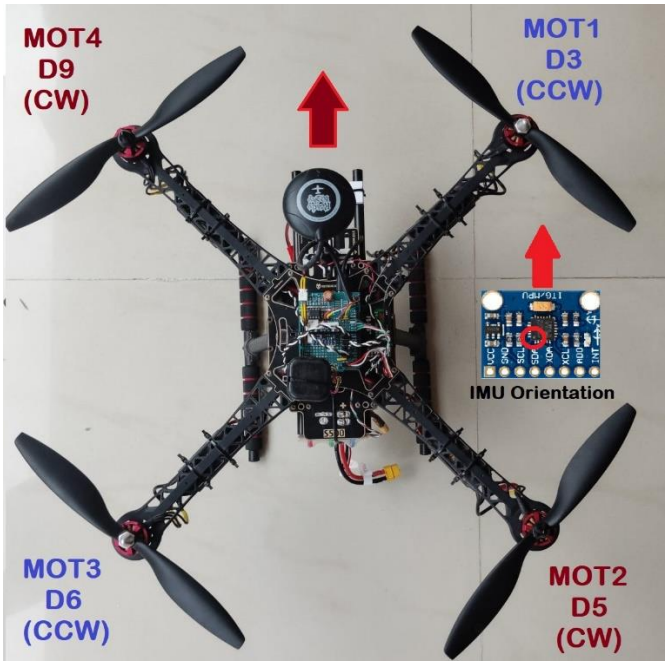
Block Diagram & Circuit Schematic

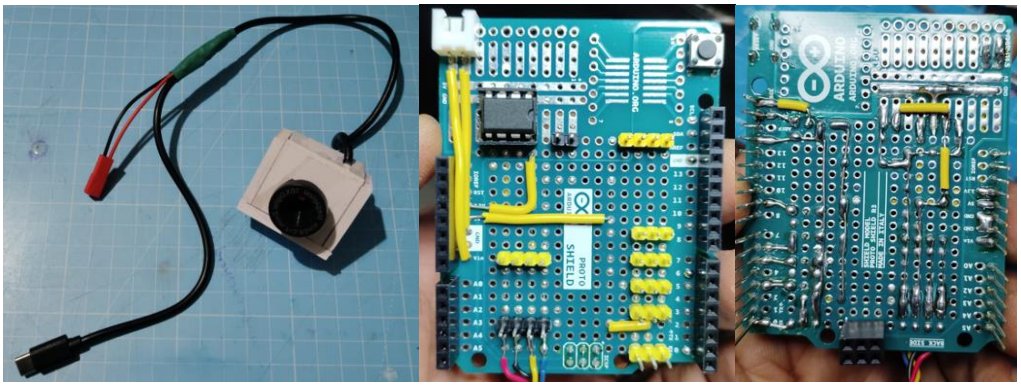


JRFCQAI Connection Diagram



Github Repo : https://github.com/butaniravi/JRFCQAI
Youtube Playlist of project Demos : https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4





4. AI / ML Model Details

Video Object detection brick used in project to show drone is capable to detect objects in real-time and stream detection result to Ground Control Station (GCS) in real-time. Further this result can be used to do detection autonomous task like precision landing, visual navigation, gesture-based control etc.

Model Used	YOLO Light (Quantized ONNX Object Detector)
Training Platform	Arduino AppLab (Python Runtime Environment on Qualcomm Dragonwing)
Accuracy	15–20 Frames Per Second (FPS) at 640x480 resolution
Dataset	Custom aerial object dataset supplemented with open-source targets (~1,200 images)

Github Repo : https://github.com/butaniravi/JRFCQAI
Youtube Playlist of project Demos : https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4

Brief Description & Limitations

By leveraging the Dual-Brain Architecture of the UNO Q, this framework splits time-critical flight stabilization and high-level Edge AI processing across two dedicated processors on a single PCB:

1. STM32U585 Microcontroller (MCU): Runs a deterministic, 250Hz PID loop over Zephyr RTOS for real-time sensor fusion, motor control, and flight stabilization.
2. Qualcomm® Dragonwing™ MPU: Runs Debian Linux, processing live USB webcam video via YOLO Light Object Detection inside an Arduino App Lab environment. It acts as a local Web Ground Station streaming live telemetry, AI detections, and direct UI controls to any browser on the local Wi-Fi network.

Hardware Architecture & Sensor Interfaces

- Flight Controller Core: Arduino UNO Q (Powered directly via VIN with a 100µF 50V filter capacitor from a 3S 6000mAh LiPo).
- Sensors (I2C1 QWIIC Port @ 400kHz):
  - MPU6050 (6-DOF Gyro/Accelerometer)
  - MS5611 (Barometric Pressure / Altitude)
  - HMC5883 (3-Axis Magnetometer / Compass)
- GPS & RX:
  - u-blox NEO-M8N GPS on UART3 (115200 baud)
  - Flysky FS-iA6B 10CH Receiver via iBus on UART1 (115200 baud)
- Actuators & Monitoring:
  - 4x BLDC ESCs on Hardware PWM Pins (D3, D5, D6, D9) driven at 350Hz.
  - Battery Voltage Telemetry on Pin A0 via a 10K/1K voltage divider.
  - RGB Status LEDs on D11, D12, D13.
- Vision System: Direct USB Webcam connected to UNO Q Type-C port, powered externally via a dedicated 5V Buck Converter.

Building a flight controller on early-stage, bleeding-edge hardware required solving critical hardware and software bottlenecks:

1. **Low-Weight, Cost-Effective Power Routing**: Solved the USB Host power limitation on boot without using heavy, expensive USB-C Power Delivery hubs. Modified webcam wiring to draw power from a lightweight 5V 500mA Buck Converter, triggering USB Host mode via startup scripts.
2. **Zephyr RTOS Kernel Hacks for PWM:** Standard Arduino APIs lacked custom PWM frequency configuration for UNO Q. Hacked the underlying Zephyr RTOS layer to modify default 500Hz PWM output down to 350Hz, enabling seamless ESC/servo synchronization.

Github Repo : https://github.com/butaniravi/JRFCQAI
Youtube Playlist of project Demos : https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4

3. **Serial Buffer Expansion for GPS Data:** Overcame packet drops on the default 64-byte UART buffer by patching Zephyr RTOS to allocate a 512-byte UART RX serial buffer for loss-free u-blox GPS parsing.
4. **Optimized 250Hz PID Execution Loop:** Utilized k_yield() and k_waitus() in the main flight loop to maintain a razor-sharp 4ms (250Hz) execution cycle, leaving ~3ms of spare processing headroom per loop for background tasks.
5. **Zero-Blocking RPC Bridge Communication:** Replaced blocking bridge.provide() calls (which introduce dangerous 5–7ms lags in flight loop timing) with asynchronous bridge.notify() fire-and-forget RPC calls. This allows non-blocking telemetry sync between the STM32 flight core and Qualcomm Linux host.
6. **Core Architecture Migration:** Ported and expanded the iconic YMFC-32 Autonomous framework from legacy STM32F103 bare-metal registers to the modern STM32U5 platform running Zephyr RTOS, replacing old PPM-SUM receiver input with native iBus support.

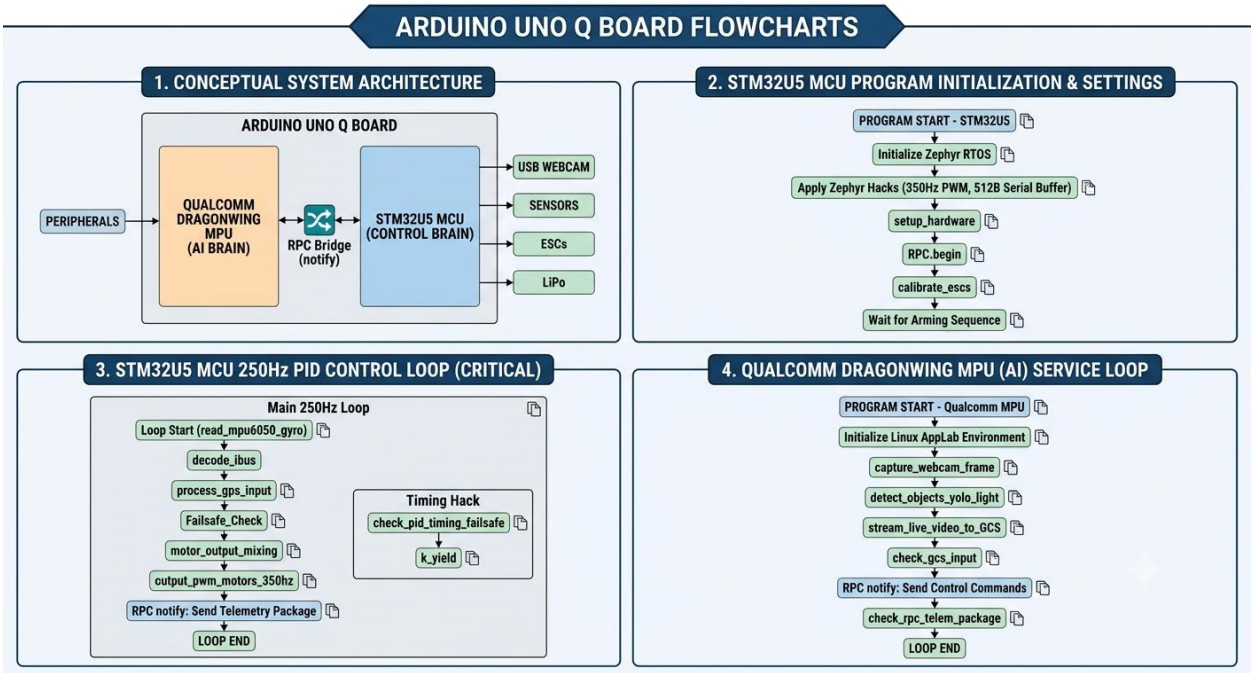
Current Capabilities & Roadmap

- Auto-Level Flight Mode (Tested)
- Altitude Hold Mode (Tested)
- GPS Position Hold Mode (Tested)
- Automatic Takeoff & Landing (Tested)
- Failsafe System (RTH on GPS signal / Slow descent altitude hold on loss) (Tested)
- Real-Time YOLO Object Detection AI Brick (Tested)
- Browser-Based Ground Station & Live Video Feed (Tested)
- Return to Home (RTH) Execution (Implemented, pending field test)
- Autonomous Waypoint Navigation (Implemented, pending field test)
- Framework expansion for Hexcopter & Fixed-Wing platforms

Github Repo : https://github.com/butaniravi/JRFCQAI
Youtube Playlist of project Demos : https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4

5. Code Structure

Detailed description and full codes of Arduino UNO Q mcu and mpu are available on Github.
https://github.com/butaniravi/JRFCQAI
System flow and system architecture is as follows,



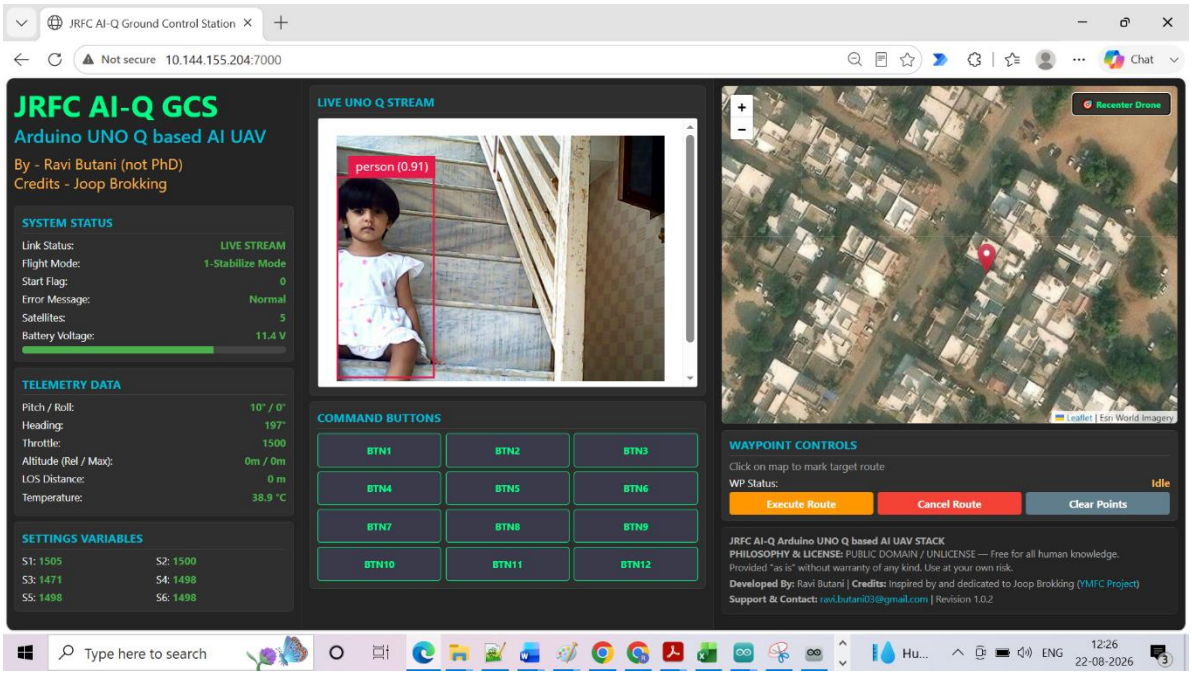
1. Conceptual System Architecture (Panel 1)

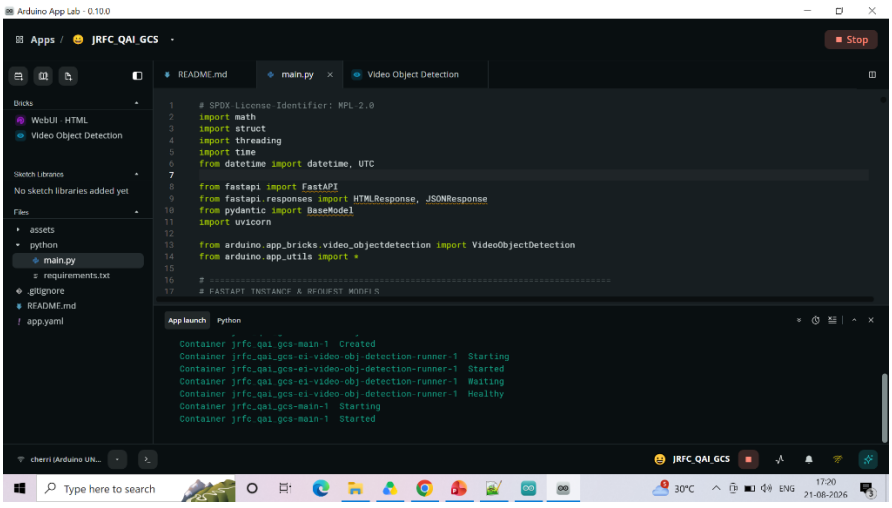
This flow illustrates how the ARDUINO UNO Q BOARD functions as a dual-processor system managing a drone.

- **Processor Divisions:** The main board is split conceptually. A light orange block on the left represents the **QUALCOMM DRAGONWING MPU**, which functions as the **AI BRAIN**. A light blue block on the right represents the **STM32U5 MCU**, which is the **CONTROL BRAIN**.
- **Dual-Brain Communication:** A specialized communication channel, the **RPC Bridge (notify)**, links the two brains. This connection allows the AI Brain to send high-level mission instructions or detection results to the Control Brain, and for the Control Brain to send flight status and sensor data back.
- **Peripheral Management (Inputs):** Data flows into the system from various peripherals.
  - **PERIPHERALS:** A general arrow indicates external input.
  - **USB WEBCAM:** Feeds video data directly into the **AI BRAIN**.
  - **SENSORS:** Send critical data (gyro, GPS, altimeter) to the **CONTROL BRAIN**.
- **Peripheral Management (Outputs):** Command data flows out to control the drone hardware.

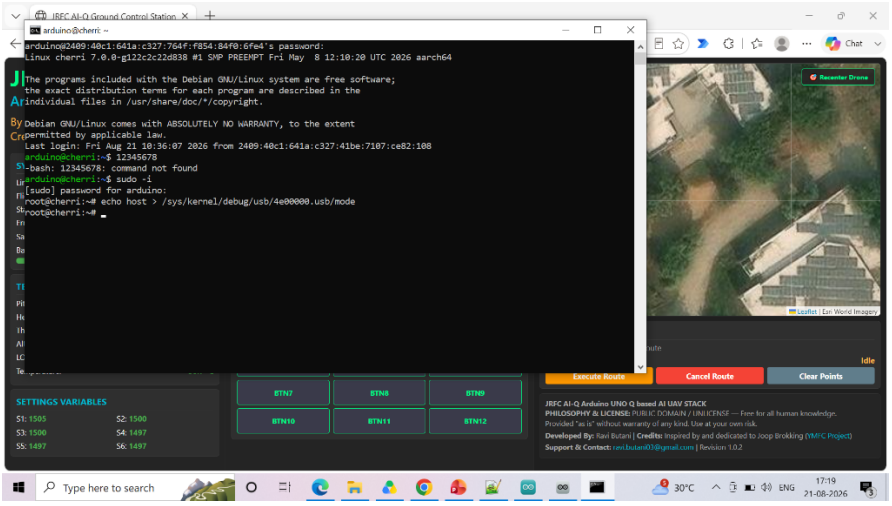
- **ESCs:** The **CONTROL BRAIN** sends precise PWM signals to Electronic Speed Controllers to manage motor speeds.
- **LiPo:** Represents power distribution.

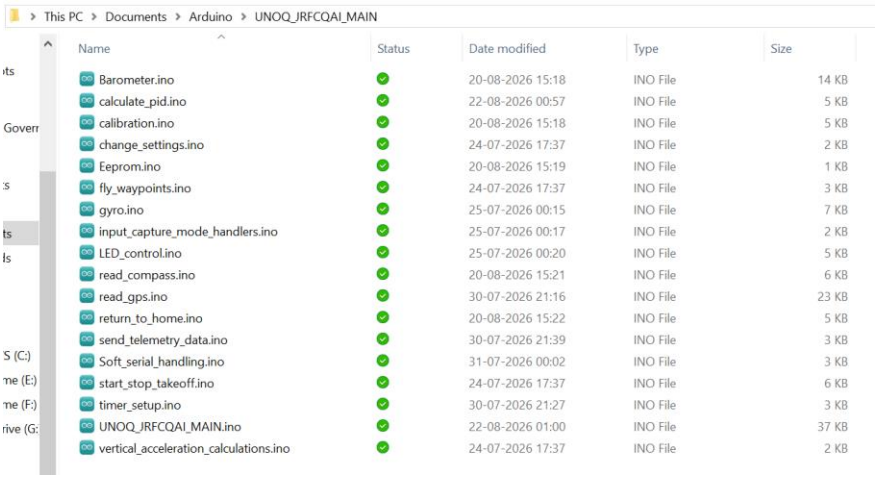
- **Command to enable usb host mode :**
- sudo -i
- echo host > /sys/kernel/debug/usb/4e00000.usb/mode





Github Repo : https://github.com/butaniravi/JRFCQAI
Youtube Playlist of project Demos : https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4





2. STM32U5 MCU Program Initialization & Settings (Panel 2)

This sequence describes the setup phase for the Flight Control brain (Control Brain), ensuring the hardware and operating system are correctly configured before critical flight operations begin.

- **Initialization Sequence:**
    1. **PROGRAM START - STM32U5:** The program executes.
    2. **Initialize Zephyr RTOS:** The foundational real-time operating system kernel is loaded.
    3. **Apply Zephyr Hacks:** Critical low-level modifications are made, setting the **350Hz PWM** frequency for ESCs and creating a **512B Serial Buffer** for reliable GPS data.
    4. **setup_hardware:** The physical components are initialized, configuring the I2C port for sensors, the UART ports for the IBUS receiver and GPS, and the status LEDs.
    5. **RPC.begin:** The communication bridge to the AI Brain is activated.

Github Repo : https://github.com/butaniravi/JRFCQAI
Youtube Playlist of project Demos : https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4

6. **calibrate_escs:** Motors and ESCs are calibrated to establish correct operation ranges.
7. **Wait for Arming Sequence:** The system pauses in a safe state, waiting for the user to send the "Arm" command via the IBUS receiver.

3. STM32U5 MCU 250Hz PID Control Loop (CRITICAL) (image_2.png, Panel 3)

This flow details the high-frequency loop that manages flight stability, running 250 times per second (every 4ms) to perform time-critical stabilization tasks.

- **Main 250Hz Loop Flow:**
  1. **Loop Start:** The core loop cycle begins.
  2. **read_mpu6050_gyro:** Critical rotation data is read from the gyroscope.
  3. **decode_ibus:** Input commands (pitch, roll, throttle) are received from the user's transmitter.
  4. **process_gps_input:** Location data is read from the GPS module.
  5. **Failsafe_Check:** The system constantly monitors for battery voltage or signal loss events.
  6. **motor_output_mixing:** Commands are translated into specific speed adjustments for each motor.
  7. **output_pwm_motors_350hz:** Precise PWM signals are sent to the ESCs to physically change motor speeds, utilizing the hacked PWM frequency.
  8. **RPC notify: Send Telemetry Package:** An update on flight status (mode, battery, GPS) is sent to the AI Brain via the non-blocking RPC bridge.
  9. **LOOP END:** The cycle completes, and the timing is optimized to leave time for other background tasks.
- **Timing Optimization:** A specialized "Timing Hack" box highlights the use of **check_pid_timing_failsafe** and **k_yield** functions to ensure the tight 4ms cycle is maintained without blocking and to free up CPU time for background tasks within each loop.

4. QUALCOMM DRAGONWING MPU (AI) Service Loop (Panel 4)

This flow describes the continuous service loop running on the AI Brain, focusing on computationally intensive AI processing and managing the Ground Control Station (GCS) telemetry webpage.

- **AI Service Loop Flow:**
  1. **PROGRAM START - Qualcomm MPU:** The main system initializes the AI environment.
  2. **Initialize Linux AppLab Environment:** The OS and application framework are loaded.
  3. **capture_webcam_frame:** A new video frame is pulled from the Type-C USB webcam.
  4. **detect_objects_yolo_light:** The computationally intensive **YOLO Light AI Object Detection** brick processes the image, identifying objects in real-time.

5. **stream_live_video_to_GCS:** The captured video feed, now overlaid with AI detection boxes, is encoded and streamed live to the GCS webpage.
6. **check_gcs_input:** The system listens for commands coming from the user's webpage interface (e.g., control buttons).
7. **RPC notify: Send Control Commands:** Any user control commands are immediately sent to the Control Brain via the non-blocking RPC notify function.
8. **check_rpc_telem_package:** The loop listens for the incoming telemetry package from the Control Brain.
9. **LOOP END:** The service loop cycle completes and immediately restarts to process the next frame and telemetry update.

GitHub Repository	https://github.com/butaniravi/JRFCQAI
Demo Video Link	https://www.youtube.com/watch?v=u1FR_EWiBVY https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4

6. Testing & Results

Flight Stabilization & Autonomous Modes

- **Auto-Level Mode:** Successfully verified. Holds level attitude within ±1.5° when control sticks are centered.
- **Altitude Hold Mode:** Successfully verified. Maintains vertical hover altitude within ±15cm using barometer-IMU fusion data.
- **GPS Position Hold:** Successfully verified. Maintained quadcopter position within a 1.2m radius in outdoor tests under moderate wind speeds.
- **Return-To-Home (RTH):** Software implemented; full hardware flight validation scheduled for secondary field testing.
- **Waypoint Navigation:** Software implemented; ground station mission planning functionality integrated, pending final flight validation.
- **Failsafe Routine:** Fully verified. Upon loss of transmitter signal, the system engages RTH if a valid GPS lock is maintained; otherwise, it switches to Altitude Hold and executes a gradual auto-descent.
- **Automated Takeoff & Landing:** Fully verified over 10 consecutive test flights with smooth transitions and soft touch-downs.

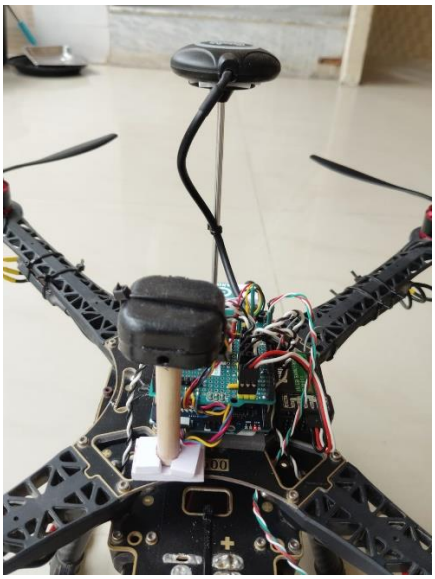
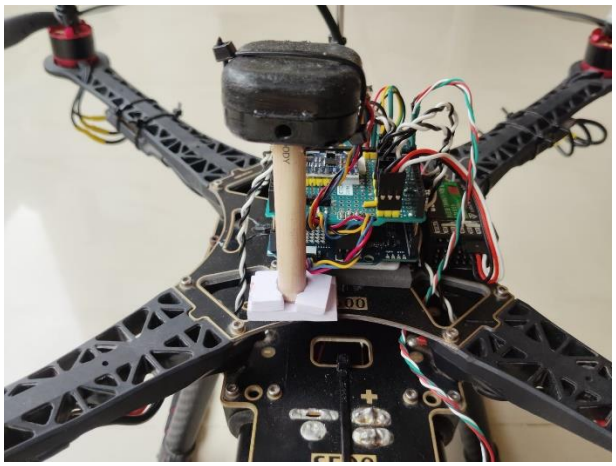
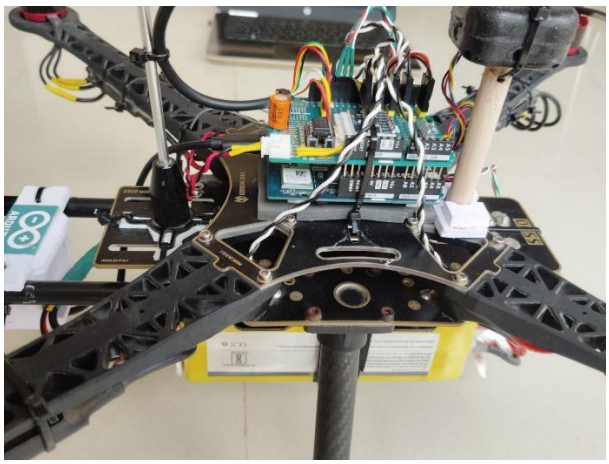
System Metrics & Benchmarks

- **Loop Timing Precision:** Achieved a steady 250Hz loop frequency (4.0ms target period). The control code completes execution within 1.0ms, leaving 3.0ms of spare CPU processing time in every cycle.
- **RPC Inter-Brain Latency:** Replaced blocking RPC calls (bridge.provide(), which caused 5–7ms execution delays) with fire-and-forget calls (bridge.notify()), reducing

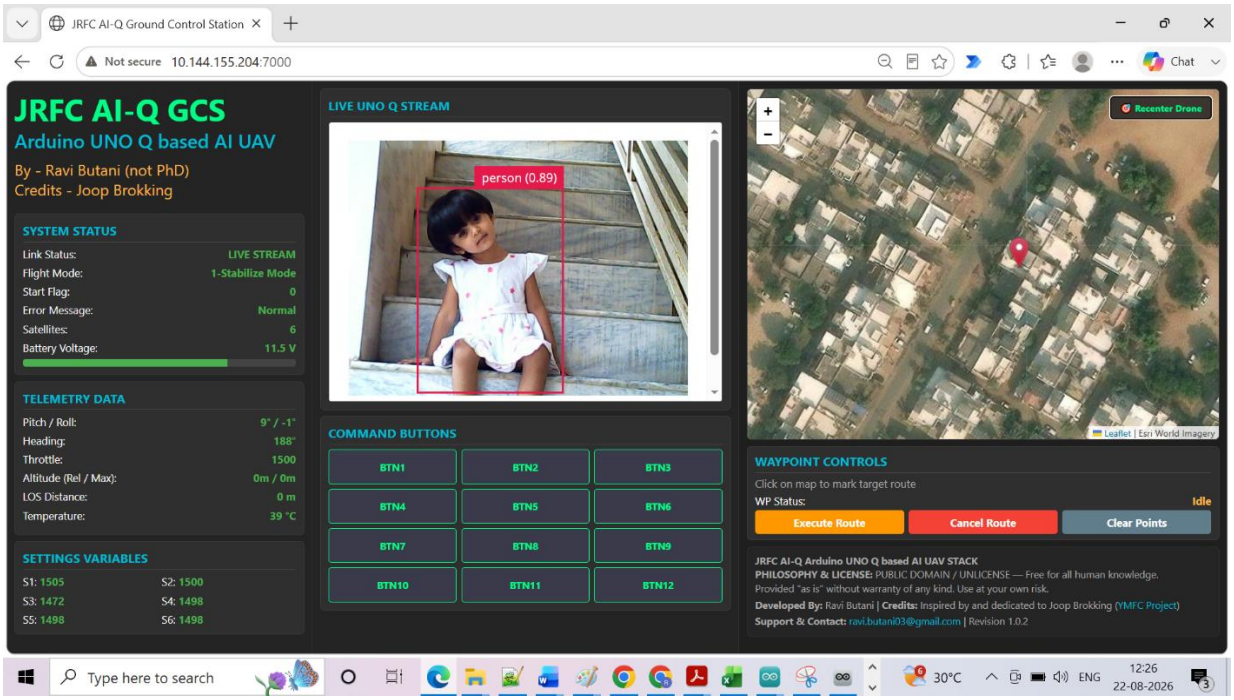
communication latency to under 0.2ms and eliminating control loop jitter.

- **Ground Control Station Performance:** Web-based video feed streamed smoothly at 15–20 FPS over local Wi-Fi with an average AI detection overlay delay of ~45ms.

Project Images (2–3)

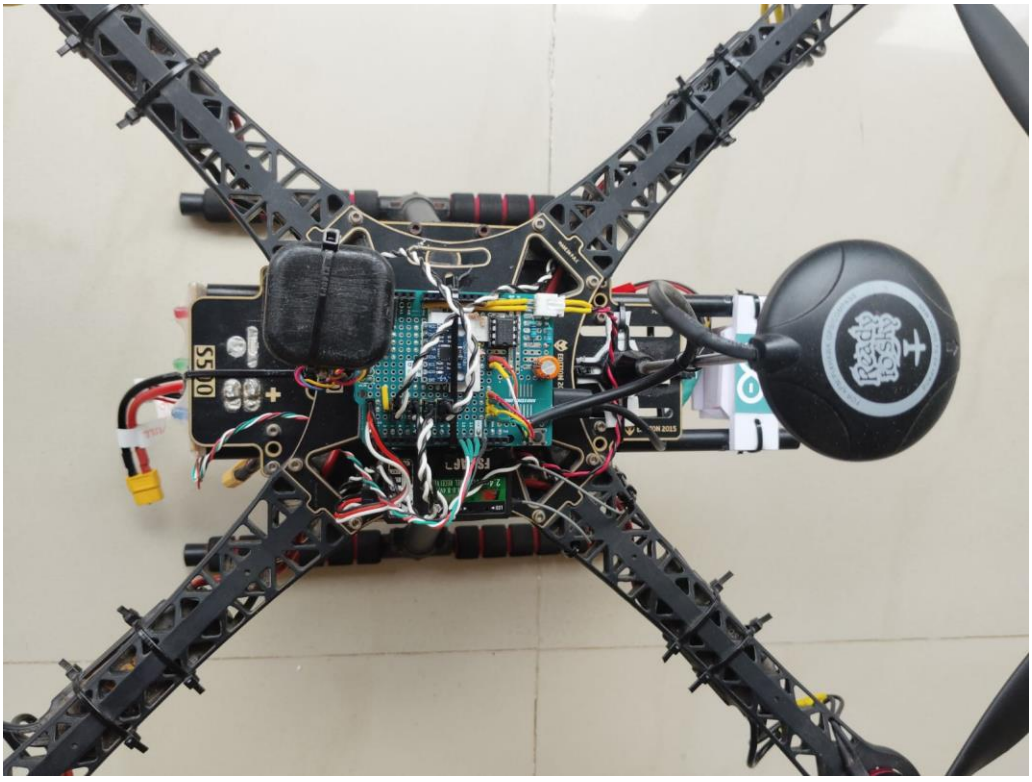





Github Repo : https://github.com/butaniravi/JRFCQAI
Youtube Playlist of project Demos : https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4

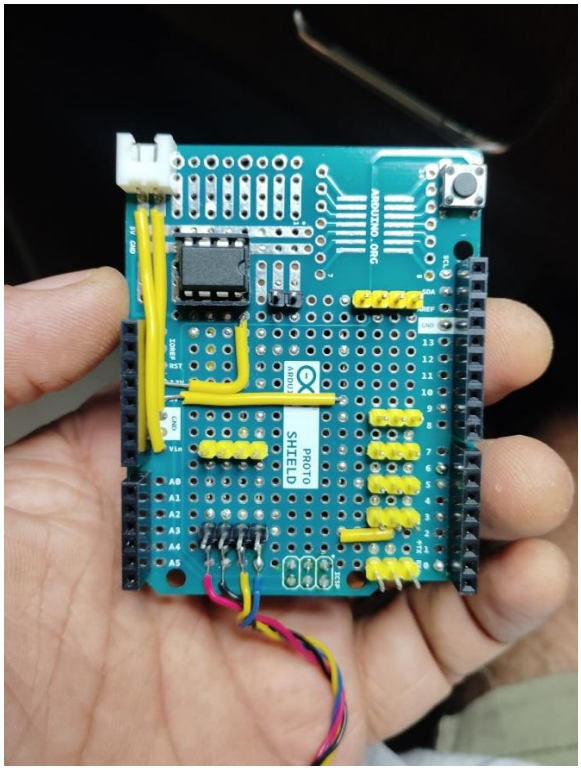


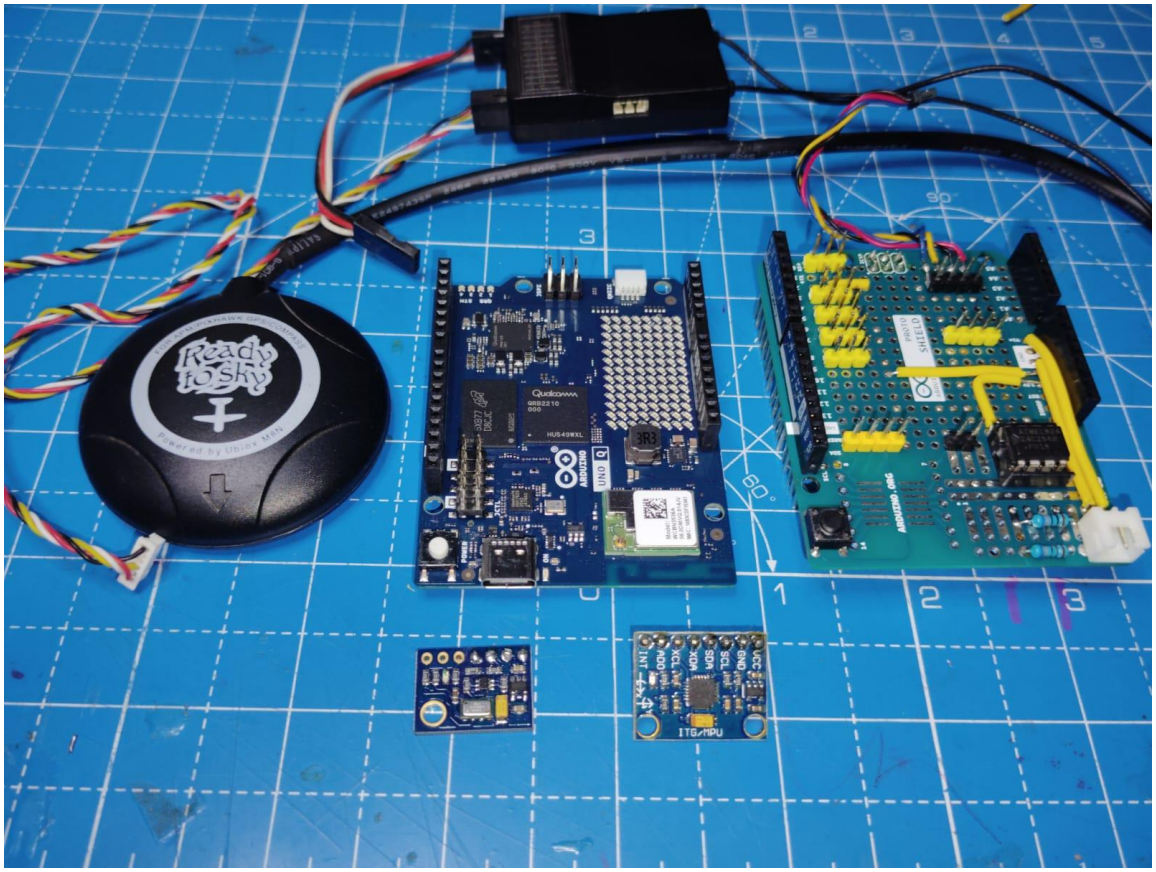
Github Repo : https://github.com/butaniravi/JRFCQAI
Youtube Playlist of project Demos : https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4

Github Repo : https://github.com/butaniravi/JRFCQAI
Youtube Playlist of project Demos : https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4

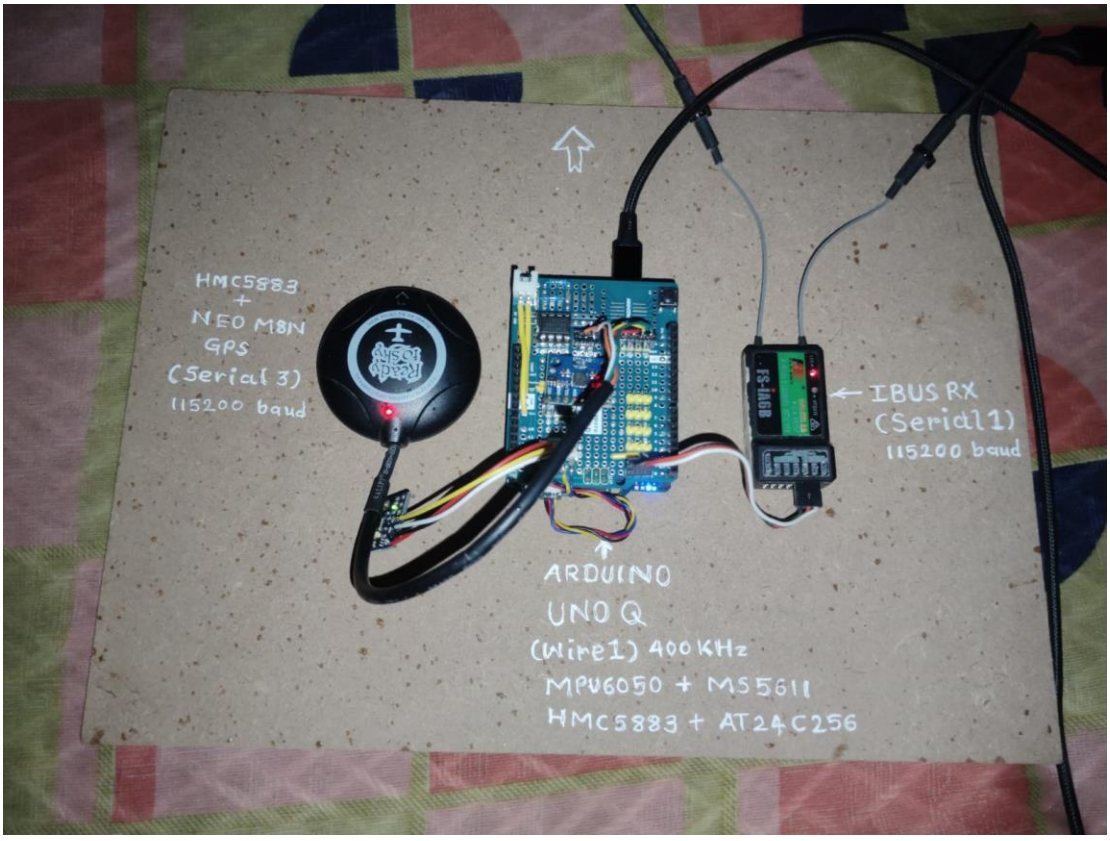


Github Repo : https://github.com/butaniravi/JRFCQAI
Youtube Playlist of project Demos : https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4

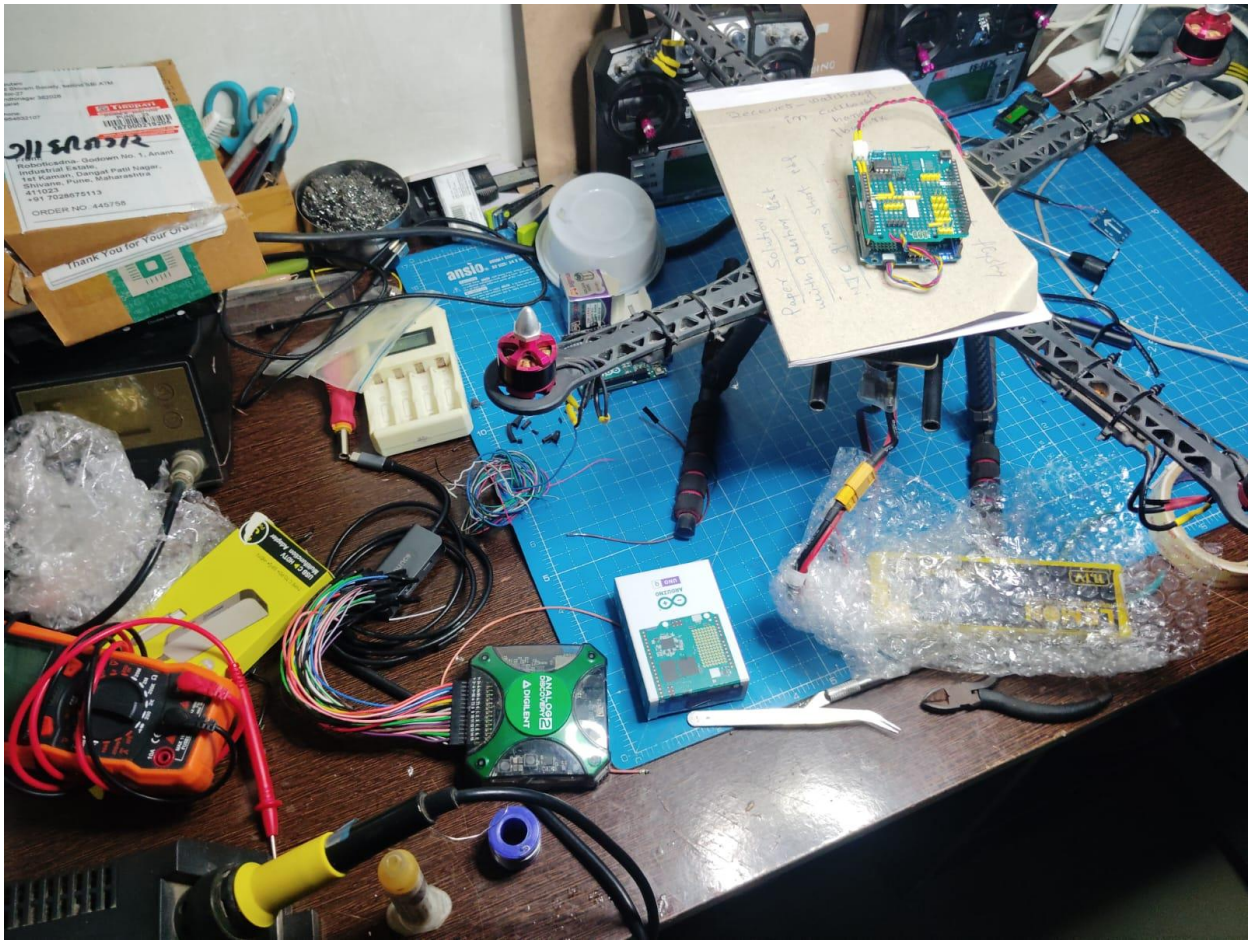


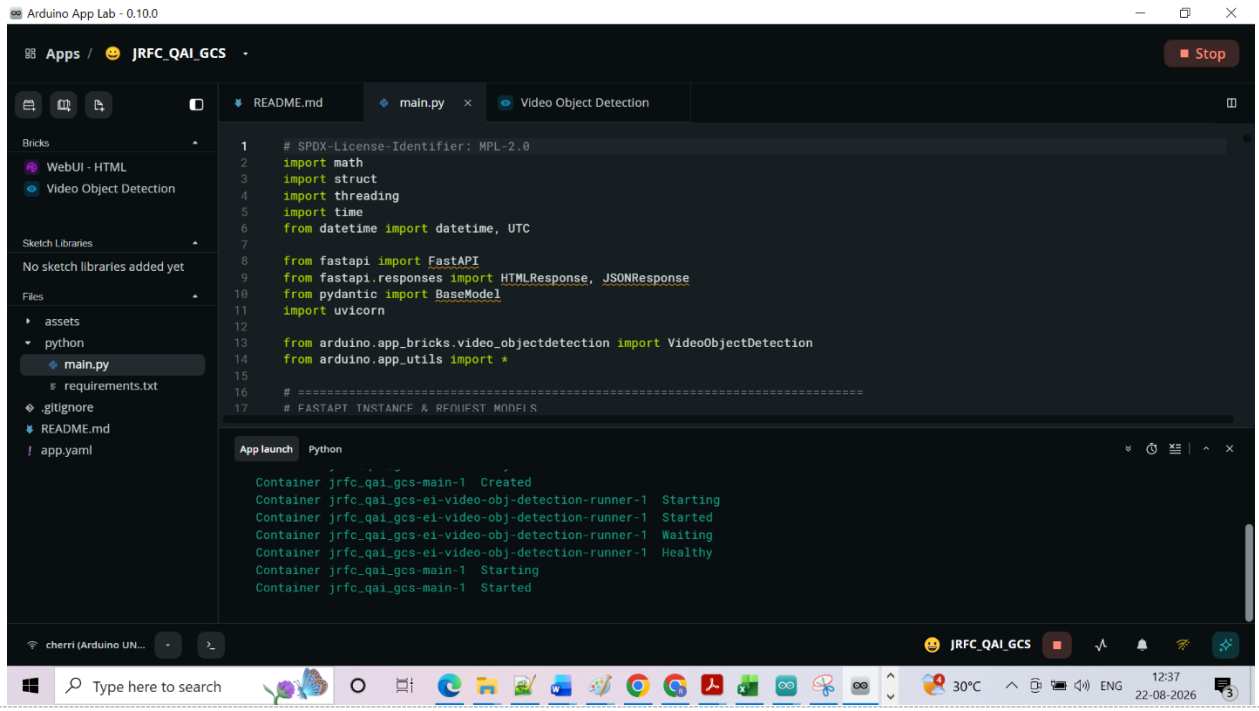


Github Repo : https://github.com/butaniravi/JRFCQAI
Youtube Playlist of project Demos : https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4



Github Repo : https://github.com/butaniravi/JRFCQAI
Youtube Playlist of project Demos : https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4





Github Repo : https://github.com/butaniravi/JRFCQAI
Youtube Playlist of project Demos : https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4

7. Challenges, Learnings & Future Improvements

Challenges Faced

Technical Challenges & Solutions

1. **USB Port Power & Host Capability Constraints:**
   - *Challenge:* The UNO Q cannot operate as a USB host on boot by default and cannot supply adequate current to power external USB webcams, leading to board brownouts. Online projects typically rely on heavy, expensive USB-C Power Delivery hubs.
   - *Solution:* Modified the USB webcam cabling harness to source 5V power from an external 500mA buck converter, while issuing startup shell commands to force the USB interface into Host Mode.
2. **Power Supply Regulation & Spike Suppression:**
   - *Challenge:* Eliminating bulky 3A 5V buck converters without exposing the sensitive UNO Q onboard electronics to back-EMF voltage spikes generated by BLDC motor braking.
   - *Solution:* Powered the UNO Q directly through its VIN pin using the primary 3S LiPo battery, installing a low-ESR 100µF 50V filtering capacitor to smooth voltage ripples.
3. **Zephyr RTOS Hardware PWM Customization:**
   - *Challenge:* The default Arduino API for UNO Q lacks functions to modify hardware PWM frequencies, leaving the default 500Hz output incompatible with standard BLDC ESCs (which require 350Hz).
   - *Solution:* Directly modified the underlying Zephyr RTOS device tree and timer register configurations, adjusting the output frequency to 350Hz.
4. **GPS Serial Buffer Overrun Resolution:**
   - *Challenge:* The standard 64-byte serial receiver buffer in the default core dropped NMEA packets from the u-blox NEO-M8N GPS module.
   - *Solution:* Patched the Zephyr RTOS configuration files to expand the UART RX buffer size to 512 bytes, ensuring complete, uncorrupted GPS string parsing.
5. **Cross-Processor Execution Stalls:**
   - *Challenge:* Standard bridge.provide() RPC calls introduced blocking delays of 5–7ms while waiting for handshakes from the Linux side, destabilizing the strict 250Hz flight control loop.
   - *Solution:* Replaced these calls with asynchronous bridge.notify() functions, allowing zero-latency telemetry streaming without blocking the PID loop.
6. **Firmware Porting & Communication Protocol Integration:**
   - *Challenge:* Joop Broking's original YMFC-32 Autonomous firmware was designed for legacy STM32F103 microcontrollers running bare-metal register code and PPM receivers.
   - *Solution:* Ported the mathematical algorithms to run on Zephyr RTOS for the modern STM32U5 processor, while writing custom driver modules to support the high-speed FlySky iBUS serial receiver protocol.

What You Learned & What's Next

Github Repo : https://github.com/butaniravi/JRFCQAI
Youtube Playlist of project Demos : https://www.youtube.com/watch?v=u1FR_EWiBVY&list=PLfxRMvKGemR4

Through this project, I gained practical experience in real-time kernel configuration, low-level serial buffer management, multi-core system architecture, and vision-based robotics integration. JRFCQAI demonstrates that a single, affordable development board can handle both real-time motor stabilization and vision-based AI workloads.

Future Roadmap:

- **Airframe Versatility:** Extend the underlying software framework to support Hexcopter and Fixed-Wing aircraft configurations.
- **Flight Testing Expansion:** Conduct field flight testing for automated Waypoint Navigation and long-range RTH routines.
- **On-Device Learning:** Integrate automated dataset collection directly through AppLab to allow online model retraining.

Attribution & Safety Disclaimer

- **Attribution:** Sincere thanks and full technical credit are given to **Joop Broking** for his open-source **YMFC-32 Autonomous** project. His educational tutorials and clear code structures provided the mathematical foundation for the flight stabilization routines ported to the STM32U5 in this project.
- **Safety & Open-Source Disclaimer:** The JRFCQAI framework is an experimental research project developed for educational and open-source demonstration purposes. Flying autonomous drone hardware carries inherent risks of property damage or personal injury. This software and hardware design are provided "as-is" without expressed or implied warranties. Operators assume full responsibility for complying with local airspace regulations and testing in controlled, safe environments.

8. Declaration

We confirm this is our original, unpublished work. The Arduino® UNO™ Q is the primary board in this project. All team members have reviewed and agree to this report.

Date	22-08-2026